

**LAPORAN PRAKTIK MATA KULIAH
DATABASE NOSQL**



KELOMPOK 1:

DHIA ATHALLA (225443006)

FARHAN MAULANA (225443008)

JOEL MARTUNAS LUMBANTORUAN (225443010)

JOSUA PARDAMEAN SIMANGUNSONG (225443011)

MUHAMAD WILDAN MUZAQI (225443014)

DOSEN PENGAMPU:

MOHAMMAD HARRY KHOMAS SAPUTRA, S.T., M.TI

1AEC-1

TEKNOLOGI REKAYASA INFORMATIKA INDUSTRI

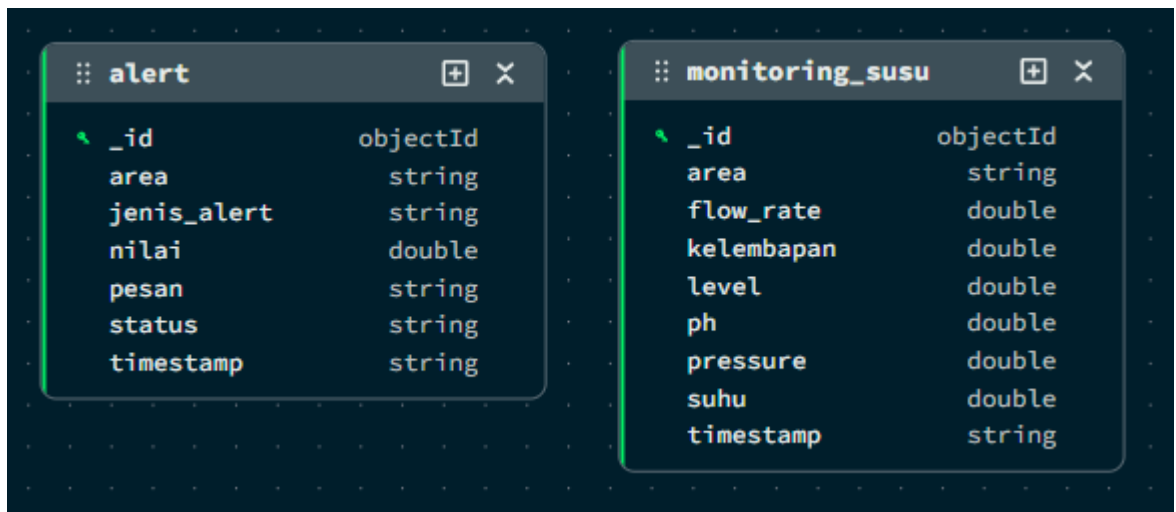
POLITEKNIK MANUFAKTUR BANDUNG

TAHUN AJARAN 2025/2026

STUDI KASUS: DASHBOARD MONITORING SUHU MESIN

DESAIN DATA

A. Desain Data



(Gambar Diagram Koleksi Database)

```
_id: ObjectId('69fafca538143c00b31a84bb')
timestamp: "2026-05-06T08:32:37.080226"
area: "Jalur Pasteurisasi"
jenis_alert: "suhu"
nilai: 50.9
status: "Bahaya"
pesan: "Suhu pasteurisasi terlalu rendah!"
```

(Gambar Contoh Dokumen Koleksi alert Database)

```
_id: ObjectId('69fafbea38143c00b31a841e')
timestamp: "2026-05-06T08:29:29.483516"
area: "Tangki Susu Utama"
suhu: 5.9
pressure: 2.6
level: 85.4
ph: 6.8
```

(Gambar Contoh Dokumen Koleksi monitoring_susu Database)

B. Generate Data Dummy

```
import random
from datetime import datetime, timedelta

try:
    from pymongo import MongoClient
    from pymongo.errors import ConnectionFailure,
    ServerSelectionTimeoutError
except ImportError:
    print("pymongo belum install")
    raise SystemExit(1)
```

```

MONGO_URI= "mongodb://localhost:27017/"
DB_NAME    = "studi_kasus_monitoring_suhu"
COL_NAME   = "suhu_mesin"

MESIN_LIST = ["Mesin_A", "Mesin_B", "Mesin_C",
              "Mesin_D", "Mesin_E"]
TOTAL_DOCS = 100
WINDOW_MIN  = 60

OVERHEAT_PROB = {
    "Mesin_A": 0.05,
    "Mesin_B": 0.30,
    "Mesin_C": 0.10,
    "Mesin_D": 0.20,
    "Mesin_E": 0.05,
}

def random_suhu(nama_mesin: str) -> float:
    if random.random() < OVERHEAT_PROB.get(nama_mesin,
0.1):
        return round(random.uniform(91.0, 105.0), 2)    #
di atas threshold
        return round(random.uniform(60.0, 89.9), 2)

def generate_docs(n: int) -> list:
    now = datetime.utcnow()
    docs = []
    for _ in range(n):
        nama = random.choice(MESIN_LIST)
        offset_sec = random.randint(0, WINDOW_MIN * 60)
        docs.append({
            "nama_mesin": nama,
            "suhu": random_suhu(nama),
            "timestamp": now -
timedelta(seconds=offset_sec),
        })
        # Urutkan by timestamp agar lebih natural
    docs.sort(key=lambda d: d["timestamp"])
    return docs

def main():
    print("=" * 50)
    print("  Data Generator - Suhu Mesin")
    print("=" * 50)

```

```

try:
    client = MongoClient(MONGO_URI,
serverSelectionTimeoutMS=3000)
    client.admin.command("ping")
except (ConnectionFailure,
ServerSelectionTimeoutError):
    print("\n[OFFLINE] Tidak bisa konek ke MongoDB.
Pastikan mongod berjalan.\n")
    return

col = client[DB_NAME][COL_NAME]
deleted = col.delete_many({}).deleted_count
if deleted:
    print(f" [INFO] {deleted} dokumen lama
dihapus.")

docs = generate_docs(TOTAL_DOCS)
result = col.insert_many(docs)
print(f" [OK] {len(result.inserted_ids)} dokumen
berhasil di-insert.")
print(f" DB : {DB_NAME}")
print(f" Koleksi: {COL_NAME}")
print(f" Periode: {WINDOW_MIN} menit terakhir
(UTC) ")

print("\n Distribusi per mesin:")
for m in MESIN_LIST:
    count = sum(1 for d in docs if d["nama_mesin"]
== m)
    alarms = sum(1 for d in docs if d["nama_mesin"]
== m and d["suhu"] > 90)
    print(f" {m:<12} : {count:>3} data |
{alarms:>2} alarm")

print("\n Jalankan monitor.py untuk melihat
dashboard.")
print("=" * 50)

if __name__ == "__main__":
    main()

```

C. Memastikan Data Siap

1. db.suhu_mesin.countDocuments()

```
> db.suhu_mesin.countDocuments()  
< 100
```

2. db.suhu_mesin.findOne()

```
> db.suhu_mesin.findOne()  
< {  
  _id: ObjectId('69fae7a61889060a285d6b49'),  
  nama_mesin: 'Mesin_D',  
  suhu: 93.55,  
  timestamp: 2026-05-06T06:03:54.560Z  
}
```

PIPELINE DAN HASIL

A. Kode Lengkap

```
import logging  
from datetime import datetime, timedelta, timezone  
from pymongo import MongoClient  
from pymongo.errors import ConnectionFailure,  
ServerSelectionTimeoutError  
  
# === 1. KONFIGURASI ===  
MONGO_URI = "mongodb://localhost:27017/"  
DB_NAME = "studi_kasus_monitoring_suhu"  
COLLECTION_NAME = "suhu_mesin"  
BATAS_ALARM = 90  
WINDOW_HOURS = 1  
  
logging.basicConfig(  
    filename="alarm.log",  
    level=logging.WARNING,  
    format="%(asctime)s | %(levelname)s | %(message)s"  
)  
  
try:
```

```

    client = MongoClient(MONGO_URI,
serverSelectionTimeoutMS=3000)

    client.admin.command("ping") # Tes koneksi
    db = client[DB_NAME]
    collection = db[COLLECTION_NAME]
except (ConnectionFailure, ServerSelectionTimeoutError):
    print("\n[OFFLINE] MongoDB tidak bisa dihubungi.
Pastikan service mongod jalan.")
    exit()

now = datetime.now(timezone.utc)
cutoff = now - timedelta(hours=WINDOW_HOURS)
filter_waktu = {"timestamp": {"$gte": cutoff}}

pipeline_latest = [
    {"$match": filter_waktu},
    {"$sort": {"timestamp": -1}},
    {
        "$group": {
            "_id": "$nama_mesin",
            "suhu_terkini": {"$first": "$suhu"},
            "timestamp": {"$first": "$timestamp"}
        }
    },
    {"$sort": {"_id": 1}}
]

pipeline_alarm = [
    {
        "$match": {
            **filter_waktu,
            "suhu": {"$gt": BATAS_ALARM}
        }
    },
    {
        "$group": {

```

```

        "_id": "$nama_mesin",
        "suhu_max": {"$max": "$suhu"},
        "timestamp": {"$max": "$timestamp"}
    }
},
{"$sort": {"suhu_max": -1}}
]

pipeline_avg = [
    {"$match": filter_waktu},
    {
        "$group": {
            "_id": "$nama_mesin",
            "avg_suhu": {"$avg": "$suhu"},
            "jumlah_data": {"$sum": 1}
        }
    },
    {"$sort": {"_id": 1}}
]

try:
    latest_data =
list(collection.aggregate(pipeline_latest))
    alarm_data =
list(collection.aggregate(pipeline_alarm))
    avg_data =
list(collection.aggregate(pipeline_avg))
except Exception as e:
    print(f"[ERROR] Terjadi masalah saat agregasi: {e}")
    exit()

if not latest_data:
    print(f"\n[INFO] Tidak ada data dalam {WINDOW_HOURS}
jam terakhir di DB: {DB_NAME}")
    exit()

```

```

print("="*60)
print(f" DASHBOARD MONITORING MESIN -
{now.strftime('%Y-%m-%d %H:%M:%S')} UTC")
print("="*60)

print("\n[1] SUHU TERKINI PER MESIN")
print("-" * 60)
print(f"{'NAMA MESIN':<15} | {'SUHU':<10} | {'WAKTU
TERAKHIR'}")
for data in latest_data:
    waktu_str = data['timestamp'].strftime('%H:%M:%S')
    print(f"{data['_id']:<15} |
{data['suhu_terkini']:>6} °C | {waktu_str}")

print("\n[2] RATA-RATA SUHU (1 JAM TERAKHIR)")
print("-" * 60)
for avg in avg_data:
    print(f"{avg['_id']:<15} | Avg:
{round(avg['avg_suhu'], 2):>6} °C | {avg['jumlah_data']}
data")

print("\n[3] STATUS ALARM (> 90°C)")
print("-" * 60)
if not alarm_data:
    print("✅ STATUS AMAN: Tidak ada mesin yang
overheat.")
else:
    for alarm in alarm_data:
        print(f"⚠️ ALARM: {alarm['_id']:<10} mencapai
{alarm['suhu_max']} °C!")
        # Catat ke file log
        logging.warning(
            f"ALARM | Mesin: {alarm['_id']} | Suhu:
{alarm['suhu_max']} °C | Waktu: {alarm['timestamp']}"
        )
        print(f"\n(Detail alarm telah dicatat ke file
alarm.log)")

```



```
print("\n" + "="*60)
print("Dashboard Selesai.\n")
```

Output:

```
=====
DASHBOARD MONITORING MESIN - 2026-05-06 07:29:24 UTC
=====

[1] SUHU TERKINI PER MESIN
-----
NAMA MESIN      | SUHU          | WAKTU TERAKHIR
Mesin_A         | 66.38 °C      | 07:02:34
Mesin_B         | 85.33 °C      | 07:00:42
Mesin_C         | 89.49 °C      | 07:00:59
Mesin_D         | 81.99 °C      | 06:50:08
Mesin_E         | 73.9 °C       | 06:57:58

[2] RATA-RATA SUHU (1 JAM TERAKHIR)
-----
Mesin_A         | Avg: 77.96 °C | 10 data
Mesin_B         | Avg: 83.55 °C | 14 data
Mesin_C         | Avg: 80.95 °C | 15 data
Mesin_D         | Avg: 80.56 °C | 5 data
Mesin_E         | Avg: 71.71 °C | 7 data

[3] STATUS ALARM (> 90°C)
-----
⚠️ ALARM: Mesin_B      mencapai 103.48 °C!
⚠️ ALARM: Mesin_C      mencapai 101.17 °C!
⚠️ ALARM: Mesin_A      mencapai 98.02 °C!
⚠️ ALARM: Mesin_D      mencapai 96.38 °C!

(Detail alarm telah dicatat ke file alarm.log)

=====
Dashboard Selesai.
```

B. Pipeline suhu terkini

```

pipeline_latest = [
    {"$match": filter_waktu},
    {"$sort": {"timestamp": -1}},
    {
        "$group": {
            "_id": "$nama_mesin",
            "suhu_terkini": {"$first": "$suhu"},
            "timestamp": {"$first": "$timestamp"}
        }
    },
    {"$sort": {"_id": 1}}
]

```

Pipeline suhu terkini digunakan untuk mengambil suhu paling baru dari setiap mesin dalam periode 1 jam terakhir. Pertama, tahap \$match digunakan untuk memfilter dokumen berdasarkan field timestamp, sehingga hanya data yang waktunya lebih besar dari waktu sekarang dikurangi satu jam yang diproses. Setelah itu, tahap \$sort mengurutkan data berdasarkan timestamp secara descending (-1), artinya data terbaru akan berada di urutan paling atas. Selanjutnya, tahap \$group mengelompokkan data berdasarkan nama_mesin. Karena data sudah diurutkan dari yang terbaru, operator \$first dapat digunakan untuk mengambil suhu dan timestamp pertama dari setiap kelompok, yang berarti data tersebut adalah data terbaru milik mesin tersebut. Terakhir, hasil diurutkan kembali berdasarkan nama mesin agar tampilan lebih rapi.

C. Pipeline alarm

```

pipeline_alarm = [
    {
        "$match": {
            **filter_waktu,
            "suhu": {"$gt": BATAS_ALARM}
        }
    },
    {
        "$group": {
            "_id": "$nama_mesin",
            "suhu_max": {"$max": "$suhu"},
            "timestamp": {"$max": "$timestamp"}
        }
    }
]

```

```

    }
  },
  {"$sort": {"suhu_max": -1}}
]

```

Pipeline alarm digunakan untuk mendeteksi mesin yang pernah memiliki suhu di atas batas alarm, yaitu 90°C, dalam 1 jam terakhir. Tahap pertama menggunakan \$match untuk memfilter data berdasarkan dua kondisi sekaligus: timestamp harus berada dalam 1 jam terakhir dan suhu harus lebih besar dari 90. Setelah data yang memenuhi syarat ditemukan, tahap \$group mengelompokkan data berdasarkan nama_mesin. Pada tahap ini digunakan operator \$max untuk mencari suhu tertinggi (suhu_max) yang pernah dicapai oleh setiap mesin dalam periode tersebut. Selain itu, timestamp terbesar juga diambil untuk mengetahui waktu alarm terbaru. Setelah proses grouping selesai, hasil diurutkan berdasarkan suhu maksimum secara descending agar mesin dengan suhu paling tinggi muncul di bagian atas.

D. Pipeline rata-rata suhu

```

pipeline_avg = [
  {"$match": filter_waktu},
  {
    "$group": {
      "_id": "$nama_mesin",
      "avg_suhu": {"$avg": "$suhu"},
      "jumlah_data": {"$sum": 1}
    }
  },
  {"$sort": {"_id": 1}}
]

```

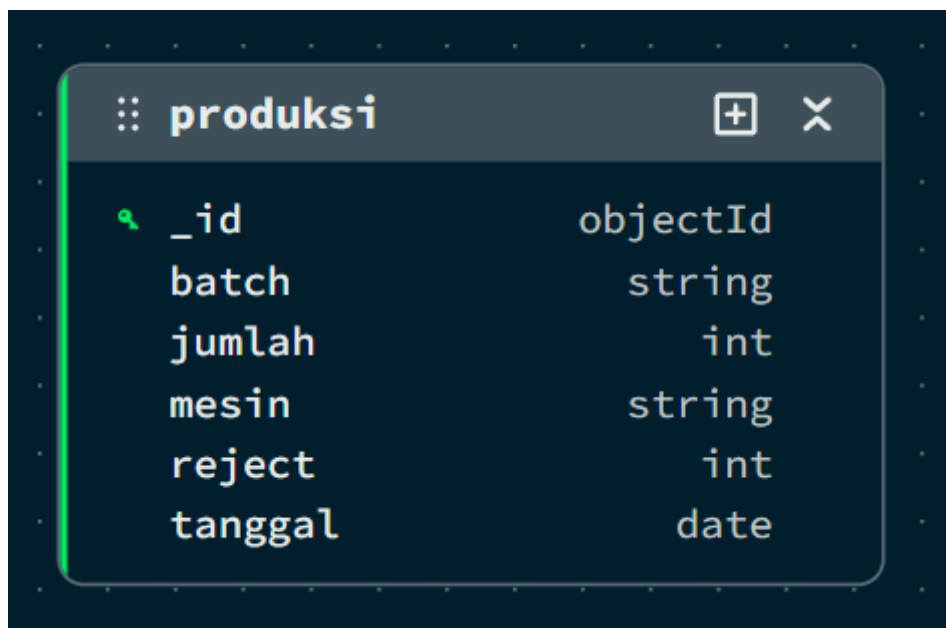
Pipeline rata-rata suhu digunakan untuk menghitung rata-rata suhu setiap mesin dalam periode monitoring 1 jam terakhir. Sama seperti pipeline sebelumnya, tahap \$match terlebih dahulu memfilter data agar hanya dokumen dalam rentang waktu tersebut yang diproses. Kemudian tahap \$group mengelompokkan data berdasarkan nama_mesin. Di dalam grouping ini digunakan operator \$avg untuk menghitung nilai rata-rata dari field suhu pada setiap kelompok mesin. Selain itu, operator \$sum: 1 digunakan untuk menghitung jumlah dokumen atau jumlah data suhu yang dimiliki masing-masing mesin selama periode monitoring. Setelah semua perhitungan selesai, tahap \$sort digunakan untuk mengurutkan hasil berdasarkan nama mesin agar output lebih mudah dibaca.

TUGAS TERSTRUKTUR (TUGAS MANDIRI)

A. Menyiapkan Folder, Virtual Enviroment dan Desain Data Database

```
PS C:\Users\asus> python -m venv venv
PS C:\Users\asus> python -m venv venv
>> .\venv\Scripts\activate
O (venv) PS C:\Users\asus>
```

Tahap awal ini bertujuan untuk mengisolasi proyek agar pustaka (*library*) yang diinstal tidak bentrok dengan proyek lain di komputer. Perintah `python -m venv venv` digunakan untuk membuat lingkungan virtual, dan `.\venv\Scripts\activate` digunakan untuk mengaktifkannya.



(Gambar Diagram Koleksi Database)

```
_id: ObjectId('69faf7904accd81aa65b2f47')
batch: "BATCH-002"
mesin: "CNC-02"
jumlah: 35
reject: 6
tanggal: 2026-12-23T00:00:00.000+00:00
```

(Gambar Contoh Dokumen Koleksi produksi Database)

B. Menghubungkan Python ke MongoDB

```
[C:\Users\user> pip install pymongo pandas python-dotenv
Collecting pymongo
  Using cached pymongo-4.17.0-cp314-cp314-win_amd64.whl.metadata (10 kB)
Collecting pandas
  Using cached pandas-3.0.2-cp314-cp314-win_amd64.whl.metadata (10 kB)
Collecting python-dotenv
  Using cached python_dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
Collecting dropython-2.8.8, >=2.8.2 (from pymongo)
  Using cached dropython-2.8.8-py3-none-any.whl.metadata (5.7 kB)
Collecting numpy-2.3.3 (from pandas)
  Using cached numpy-2.4.4-cp314-cp314-win_amd64.whl.metadata (5.6 kB)
Collecting python-dateutil-2.9.0.post0 (from pandas)
  Using cached python_dateutil-2.9.0.post0-py3-none-any.whl.metadata (8.4 kB)
Collecting tzdata (from pandas)
  Using cached tzdata-2025.2-py3-none-any.whl.metadata (1.4 kB)
Collecting six-1.17.0 (from python-dateutil-2.9.0.post0)
  Using cached six-1.17.0-py2.py3-none-any.whl.metadata (1.7 kB)
Using cached pymongo-4.17.0-cp314-cp314-win_amd64.whl (1.0 MB)
Using cached dropython-2.8.8-py3-none-any.whl (331 kB)
Using cached pandas-3.0.2-cp314-cp314-win_amd64.whl (10.9 MB)
Using cached python_dotenv-1.2.2-py3-none-any.whl (22 kB)
Using cached numpy-2.4.4-cp314-cp314-win_amd64.whl (12.4 MB)
Using cached python_dateutil-2.9.0.post0-py3-none-any.whl (220 kB)
Using cached six-1.17.0-py2.py3-none-any.whl (11 kB)
Using cached tzdata-2025.2-py3-none-any.whl (100 kB)
Installing collected packages: tzdata, six, python-dotenv, numpy, dropython, python-dateutil, pymongo, pandas
Successfully installed dropython-2.8.8 numpy-2.4.4 pandas-3.0.2 pymongo-4.17.0 python-dateutil-2.9.0.post0 python-dotenv-1.2.2 six-1.17.0 tzdata-2025.2

[notice] A new release of pip is available: 26.0.1 -> 26.3.1
[notice] To update, run: python.exe -m pip install --upgrade pip
user@C:\Users\user>
```

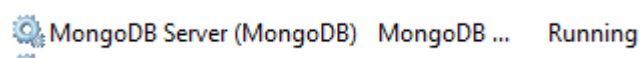
Pada tahap ini, dilakukan instalasi pustaka eksternal yang dibutuhkan sistem, yaitu pymongo untuk koneksi database, pandas untuk pengolahan tabel data, dan python-dotenv untuk membaca file konfigurasi rahasia.

C. Buat File Baru dengan Nama .env

```
.env
1 MONGO_URI=mongodb://localhost:27017
2 DB_NAME=latihan_industri
```

.env digunakan untuk menyimpan variabel sensitif seperti alamat database (MONGO_URI) dan nama database (DB_NAME). Hal ini bertujuan agar kode program lebih aman dan mudah dikonfigurasi tanpa harus mengubah isi skrip Python utama.

D. Run Mongo Db Server



*karena kode yang di buat `client.admin.command('ping')` yang akan langsung menghentikan program jika database tidak merespons. Sebelum menjalankan aplikasi, layanan *MongoDB Server* harus dipastikan dalam status *Running* di Windows Services. Hal ini krusial karena kode program menggunakan fungsi ping yang akan secara otomatis menghentikan program jika database tidak memberikan respons.

E. Make Python and Run

```

32     client.admin.command('ping') # Yes apakah database benar-benar nyala
33     db = client[DB_NAME]
34     collection = db['produksi']
35     logging.info(f'Koneksi Berhasil! Menggunakan Database: {DB_NAME}')
36 except Exception as e:
37     print("\nX ERROR: Gagal terhubung ke MongoDB.")
38     print("Pastikan aplikasi MongoDB Server sudah 'Running' di Services.")
39     logging.error(f'Koneksi DB Gagal: {e}')
40     exit()
41
42 # =====
43 # 3. FUNGSI-FUNGSI SISTEM
44 # =====
45
46 def input_data():
47     print("\n" + "-"*30)
48     print("1. INPUT DATA PRODUKSI BARU")
49     print("-"*30)
50     try:
51         batch = input("Masukkan Batch (contoh: BATCH-001): ").upper()
52         mesin = input("Masukkan Mesin (contoh: CNC-01): ").upper()
53         jumlah = int(input("Masukkan Jumlah Produksi: "))
54         reject = int(input("Masukkan Jumlah Reject: "))
55         tanggal_str = input("Masukkan Tanggal (YYYY-MM-DD): ")
56
57         # Validasi format tanggal
58         tanggal = datetime.strptime(tanggal_str, "%Y-%m-%d")
59
60         doc = {
61             "batch": batch,
62             "mesin": mesin,
63             "jumlah": jumlah,

```

```

32     client.admin.command('ping') # Yes apakah database benar-benar nyala
33     db = client[DB_NAME]
34     collection = db['produksi']
35     logging.info(f'Koneksi Berhasil! Menggunakan Database: {DB_NAME}')
36 except Exception as e:
37     print("\nX ERROR: Gagal terhubung ke MongoDB.")
38     print("Pastikan aplikasi MongoDB Server sudah 'Running' di Services.")
39     logging.error(f'Koneksi DB Gagal: {e}')
40     exit()
41
42 # =====
43 # 3. FUNGSI-FUNGSI SISTEM
44 # =====
45
46 def input_data():
47     print("\n" + "-"*30)
48     print("1. INPUT DATA PRODUKSI BARU")
49     print("-"*30)
50     try:
51         batch = input("Masukkan Batch (contoh: BATCH-001): ").upper()
52         mesin = input("Masukkan Mesin (contoh: CNC-01): ").upper()
53         jumlah = int(input("Masukkan Jumlah Produksi: "))
54         reject = int(input("Masukkan Jumlah Reject: "))
55         tanggal_str = input("Masukkan Tanggal (YYYY-MM-DD): ")
56
57         # Validasi format tanggal
58         tanggal = datetime.strptime(tanggal_str, "%Y-%m-%d")
59
60         doc = {
61             "batch": batch,
62             "mesin": mesin,
63             "jumlah": jumlah,

```

```

64         "reject": reject,
65         "tanggal": tanggal
66     }
67
68     collection.insert_one(doc)
69     print("\n✅ Data berhasil disimpan ke MongoDB!")
70     logging.info(f"Insert Berhasil: {batch} | Mesin: {mesin}")
71
72 except ValueError:
73     print("\n❌ Input Salah! Pastikan Jumlah/Reject berupa angka dan
74 except Exception as e:
75     print(f"\n❌ Gagal menyimpan: {e}")
76
77 def tampilkan_data_mesin():
78     print("\n" + "-"*30)
79     print("2. TAMPILKAN DATA PER MESIN")
80     print("-"*30)
81     mesin = input("Masukkan Nama Mesin yang dicari: ").upper()
82     try:
83         cursor = collection.find({"mesin": mesin}, {"_id": 0})
84         data = list(cursor)
85
86         if not data:
87             print(f"⚠️ Tidak ditemukan data untuk mesin {mesin}.")
88             return
89
90         df = pd.DataFrame(data)
91         df['tanggal'] = df['tanggal'].dt.strftime('%d-%m-%Y')
92         print("\n", df.to_string(index=False))
93

```

```

77 def tampilkan_data_mesin():
78     print(f"❌ Terjadi kesalahan: {e}")
79
80 def hitung_reject_rate():
81     print("\n" + "-"*30)
82     print("3. ANALISIS BATCH REJECT TINGGI (>5%)")
83     print("-"*30)
84
85     pipeline = [
86         {"$match": {"jumlah": {"$gt": 0}}},
87         {"$project": {
88             "_id": 0,
89             "batch": 1,
90             "mesin": 1,
91             "jumlah": 1,
92             "reject": 1,
93             "reject_rate": { "$multiply": [{"$divide": [{"$reject", "$jum
94         }],
95         {"$match": {"reject_rate": {"$gt": 5}}},
96         {"$sort": {"reject_rate": -1}}
97     ]
98
99     try:
100         results = list(collection.aggregate(pipeline))
101         if not results:
102             print("✅ Semua batch memiliki reject rate di bawah 5%")
103             return
104
105         df = pd.DataFrame(results)
106         df['reject_rate'] = df['reject_rate'].round(2).astype(str) + '%'
107         print("\n--- DAFTAR BATCH BERMASALAH ---")
108         print(df.to_string(index=False))
109

```

```

127 except Exception as e:
128     print(f"❌ Gagal menghitung agregasi: {e}")
129
130 def ekspor_laporan():
131     print("\n" + "-"*30)
132     print("4. EKSPOR LAPORAN KE CSV")
133     print("-"*30)
134     filename = "laporan_produksi_full.csv"
135     try:
136         cursor = collection.find({}, {"_id": 0})
137         data = list(cursor)
138
139         if not data:
140             print("⚠️ Database kosong, tidak ada yang bisa diekspor.")
141             return
142
143         df = pd.DataFrame(data)
144         df.to_csv(filename, index=False)
145         print(f"✅ Berhasil! File tersimpan sebagai: {filename}")
146         logging.info(f"Ekspor Sukses: {filename}")
147
148 except Exception as e:
149     print(f"❌ Gagal ekspor: {e}")
150
151 # =====
152 # 4. MENU UTAMA
153 # =====
154 def main():
155     while True:
156         print("\n" + "-"*40)
157         print("SISTEM MANAJEMEN PRODUKSI INDUSTRI")
158

```

```

158     print("-"*40)
159     print("1. Input Data Baru")
160     print("2. Cari Data per Mesin")
161     print("3. Cek Reject Rate (>5%)")
162     print("4. Ekspor Semua Data ke CSV")
163     print("5. Keluar")
164     print("-"*40)
165
166     pilih = input("Pilih Menu (1-5): ")
167
168     if pilih == '1': input_data()
169     elif pilih == '2': tampilkan_data_mesin()
170     elif pilih == '3': hitung_reject_rate()
171     elif pilih == '4': ekspor_laporan()
172     elif pilih == '5':
173         print("Menutup aplikasi... Sampai jumpa!")
174         break
175     else:
176         print("Pilihan tidak tersedia!")
177
178 if __name__ == "__main__":
179     main()
180

```

Struktur kode pada file manajemen_produksi.py dirancang secara modular dan sistematis, diawali dengan konfigurasi logging yang berfungsi mencatat setiap kejadian aplikasi ke dalam file app.log secara kronologis. Sistem kemudian membangun koneksi ke database menggunakan fungsi load_dotenv() untuk mengambil pengaturan dari file .env, lengkap dengan fitur *fallback* atau cadangan alamat database agar program tidak mengalami kegagalan sistem (*crash*) jika file konfigurasi tersebut tidak ditemukan. Untuk menjaga keamanan operasional, setiap fungsi kritis seperti input data produksi, pencarian data per mesin, hingga ekspor laporan bulanan telah dibungkus dengan blok *try-except* guna menangani kesalahan input pengguna atau gangguan koneksi secara mandiri. Selain itu, kode ini juga mengimplementasikan teknik agregasi tingkat lanjut untuk menghitung persentase kerusakan barang secara otomatis di sisi server database, serta menggunakan library Pandas untuk menyajikan data dalam bentuk tabel yang rapi sebelum akhirnya diekspor ke format CSV.

HASIL:

```
=====
SISTEM MANAJEMEN PRODUKSI INDUSTRI
=====
1. Input Data Baru
2. Cari Data per Mesin
3. Cek Reject Rate (>5%)
4. Ekspor Semua Data ke CSV
5. Keluar
=====
Pilih Menu (1-5): █
```

```
=====
1. INPUT DATA PRODUKSI BARU
=====
Masukkan Batch (contoh: BATCH-001): BATCH-01
Masukkan Mesin (contoh: CNC-01): CNC-01
Masukkan Jumlah Produksi: 100
Masukkan Jumlah Reject: 2
Masukkan Tanggal (YYYY-MM-DD): 2026-05-07

✅ Data berhasil disimpan ke MongoDB!
2026-05-07 05:35:58,211 - INFO - Insert Berhasil: BATCH-01 | Mesin: CNC-01
```

Berdasarkan implementasi sistem pada file manajemen_produk.py, dapat dianalisis bahwa integrasi antara Python dan MongoDB telah berhasil membentuk sebuah ekosistem manajemen data industri yang terstruktur. Penggunaan database NoSQL MongoDB memberikan fleksibilitas dalam penyimpanan dokumen produksi yang mencakup atribut batch, mesin, hingga jumlah reject tanpa terikat skema yang kaku. Melalui fungsi agregasi yang diterapkan pada menu analisis reject rate, sistem mampu melakukan komputasi kompleks di sisi server untuk menyaring data batch yang memiliki tingkat kerusakan di atas 5%, yang mana hal ini sangat krusial untuk pengambilan keputusan cepat di rantai produksi. Selain itu, penerapan fitur ekspor data ke format CSV menunjukkan kemampuan sistem dalam mendukung interoperabilitas data, sehingga informasi dari database dapat diolah lebih lanjut menggunakan perangkat lunak perkantoran seperti Excel. Keamanan operasional juga terjaga melalui penerapan blok *try-except* pada setiap fungsi kritis, yang memastikan aplikasi tetap berjalan meskipun terjadi kesalahan input atau gangguan koneksi. Seluruh aktivitas ini terekam secara kronologis dalam file app.log, yang membuktikan bahwa prinsip *error handling* dan *logging* telah diterapkan sesuai standar pengembangan perangkat lunak industri untuk memudahkan proses pelacakan masalah di masa mendatang. Sebagai kesimpulan, sistem ini tidak hanya berfungsi sebagai alat input data, tetapi juga sebagai instrumen analisis yang mampu memberikan gambaran performa mesin secara nyata. Keberhasilan program dalam

mengelola data secara *end-to-end*—mulai dari input, penyimpanan, analisis agregasi, hingga pelaporan—menunjukkan bahwa pemanfaatan teknologi database modern dan pemrograman modular merupakan solusi efektif dalam mendigitalisasi proses manufaktur tradisional.

TUGAS KELOMPOK (3-4 ORANG)